מערכת ניתוח ותעדוף להתרעות מצלמות גבול

סיפור כיסוי

בתחילת שנת 2026 זוהתה עלייה חדה בכמות ההתראות המתקבלות ממצלמות תצפית הפרוסות לאורך גבולות ישראל — מצרים, לבנון, עזה, סוריה וירדן.

ההתראות אינן אחידות ברמת הסיכון שלהן:
חלקן מצביעות על תנועה אזרחית שגרתית, בעוד אחרות עשויות להעיד על חדירה עוינת, הברחת אמצעי לחימה או פעילות מבצעית מתוכננת.

המערכת הקיימת קולטת את ההתראות אך אינה מבצעת תיעדוף חכם ואינה מבדילה כראוי בין אירועים שגרתיים לבין אירועים דחופים הדורשים תגובה מיידית.

יחידת המודיעין הטכנולוגי נדרשת לבנות מערכת חדשה שתדע:

- לקלוט התרעות מהשטח
- לנתח אותן
- לקבוע רמת עדיפות
- לעבד אותן לפי סדר עדיפות
- ולאפשר לצוותי הביטחון גישה לנתונים מעובדים לצורך המשך חקירה

המערכת אינה מקבלת מזהה ייחודי לכל התרעה מראש.
מזהה ייווצר אוטומטית על ידי MongoDB בעת הכנסת הרשומה למסד הנתונים.

מטרת המערכת וסקירה כללית

עליכם לבנות מערכת מבוזרת מבוססת Redis ו-MongoDB לניהול התרעות מבצעיות המתקבלות ממצלמות גבול. המערכת אחראית לקלוט התרעות מקובץ JSON, לנתח כל התרעה על פי לוגיקה מוגדרת מראש, לקבוע את רמת העדיפות שלה, ולנהל את עיבודה בהתאם לסדר עדיפויות מבצעי.

עזרים מותרים

במהלך המבחן **חל איסור מוחלט** על:

- שימוש ב־**Google** או כל מנוע חיפוש שאינו כלול באחד מן המקורות המותרים
- שימוש ב־**כלי AI מכל סוג**
  (כולל אך לא מוגבל ל־ChatGPT, Copilot, Claude, Gemini וכדומה)
- קבלת עזרה מאדם אחר

✅ עזרים מותרים

מותר להשתמש **אך ורק** במקורות הבאים:

Python

- תיעוד רשמי:
  Python Docs
  PyMongo - Documentation

MongoDB

- תיעוד רשמי:
  MongoDB Documentation - Homepage

MySQL

- MySQL Documentation

Kafka

- confluent_kafka API — confluent-kafka 2.13.0 documentation
- (Kafka Crash Course (Nana
  https://gitlab.com/twn-youtube/kafka-crash-course/-/blob/main/README.md?ref_type=heads

Docker

- תיעוד רשמי:
  Docker Docs
- Docker Hub (Images, Tags, Usage):
  Docker Hub

FastAPI

- תיעוד רשמי:
  FastAPI

Git

- תיעוד רשמי:
  Git - Reference

YAML

- YAML Lint (בדיקת תקינות קבצי YAML):
  The YAML Validator

Redis

- Redis Docs
- redis-py guide (Python) | Docs
- /https://redis.readthedocs.io/en/stable

אתרי עזר כלליים

- W3Schools:
  W3 Schools
- GeeksforGeeks:
  GeeksforGeeks
- Stack Overflow:
  https://stackoverflow.com

הארכיטקטורה מבוססת תורים ומופרדת לשני רכיבי עיבוד עיקריים:

רכיב ה- Producer

האחראי על טעינת הנתונים, חישוב ה-priority והזרקת ההתרעות לתור המתאים ב-Redis (Urgent / Normal),
ורכיב Consumer, האחראי על משיכת ההתרעות לפי סדר עדיפות ושמירתן במסד נתונים MongoDB לצורכי תיעוד היסטורי.

רכיב ה- Redis

משמש כשכבת ניהול תיעדוף ועומסים בזמן אמת, ומבטיח שעיבוד התרעות דחופות יתבצע לפני רגילות.
MongoDB משמש כשכבת אחסון מתמשכת, אליה נשמרות כלל ההתרעות יחד עם זמן הכנסתן למערכת (insertion_time).

המערכת צריכה להבטיח:

- עיבוד התרעות דחופות לפני רגילות.
- עבודה רציפה תחת עומס.
- הפרדה ברורה בין שכבת הקליטה והתעדוף לבין שכבת האחסון.
- שמירה מלאה של היסטוריית ההתרעות לצורך הרחבה עתידית לשכבת אנליטיקה.

מבנה הדאטה (קלט)

המערכת תקבל קובץ JSON המכיל מערך של התרעות.

כל התרעה כוללת את השדות הבאים:

- `border` – אחד מהבאים: egypt / lebanon / gaza / syria / jordan
- `zone` – שם מזהה אזור פנימי בגבול
- `timestamp` – זמן זיהוי האירוע בפורמט ISO 8601
- `People_count` – מספר אנשים שזוהו
- `weapons_count` – מספר כלי נשק שזוהו
- `vehicle_type` – סוג רכב (motorcycle / jeep / truck / car / none)
- `distance_from_fence_m` – מרחק מהגדר במטרים
- `visibility_quality` – איכות ראות (ערך מספרי בטווח 0–1 או 0–100 בהתאם להגדרה)

חשוב:

- אין להעביר alert_id בקלט.
- מזהה ייווצר אוטומטית על ידי MongoDB בעת הכנסת הרשומה.
- אין לשנות שמות שדות.
- אין להוסיף שדות בקלט.

בנוסף, חלק מדרישות המבחן הוא להבין את מבנה הדאטה באופן עצמאי.
עליכם לזהות את סוג הנתונים (Data Types) של כל שדה בקובץ ה-JSON ולטפל בו בהתאם במימוש הלוגיקה והאנליטיקה.

יש להקפיד על עבודה נכונה עם טיפוסים – למשל השוואות מספריות יבוצעו על ערכים מספריים, טיפול בזמנים יתבצע בהתאם לפורמט התאריך, וטקסט יטופל כמחרוזת.

לא יינתנו הבהרות נוספות לגבי משמעות השדות או סוגיהם מעבר למה שמופיע במסמך זה, ועליכם להסיק זאת ישירות מהדאטה.

רכיב ה-Producer – שכבת קליטה ותעדוף

רכיב ה-Producer מהווה את נקודת הכניסה הלוגית של המערכת. תפקידו לקלוט את קובץ ההתרעות, לעבד כל התרעה באופן עצמאי, ולסווג אותה לרמת עדיפות מבצעית בהתאם לכללים שיוגדרו בהמשך המסמך.

הרכיב אחראי על מעבר סדרתי על ההתרעות המופיעות בקובץ ה-JSON, וביצוע ניתוח מבצעי לכל התרעה על בסיס הנתונים הגולמיים שלה. במסגרת תהליך זה, לכל התרעה מתווסף שדה חדש בשם `priority`, אשר מייצג את רמת הדחיפות המבצעית שנקבעה לה.

לאחר קביעת העדיפות, ההתרעה מוזרקת ל-Redis אל התור המתאים:

- `urgent_queue` – עבור התרעות המסווגות כדחופות
- `normal_queue` – עבור התרעות המסווגות כרגילות

ההזרקה מתבצעת פריט אחר פריט, תוך שמירה על סדר ההגעה המקורי של ההתרעות בתוך כל רמת עדיפות. בשלב זה לא מתבצעת כל כתיבה ל-MongoDB, ואין עקיפה של מנגנון התורים. כל התרעה חייבת לעבור דרך Redis כחלק ממנגנון התיעדוף והאיזון של המערכת.

רכיב זה אחראי אך ורק על קליטה, ניתוח וקביעת עדיפות. הוא אינו מבצע אחסון מתמשך ואינו מתקשר ישירות עם מסד הנתונים.

לוגיקת קביעת עדיפות (Priority Classification Rules)

קביעת רמת העדיפות תתבצע בצד ה-Producer, לפני הכנסת ההתרעה לתור ב-Redis. לכל התרעה יש להוסיף שדה `priority`, שערכו יהיה אחד משניים בלבד:

- `URGENT`
- `NORMAL`

הסיווג יתבצע על פי הכללים הבאים:

תנאים קריטיים (מספיק תנאי אחד)

אם מתקיים אחד מהתנאים הבאים, ההתרעה תסווג כ-`URGENT`:

- `weapons_count > 0`
- `distance_from_fence_m <= 50`
- `people_count >= 8`
- `"vehicle_type == "truck`

תנאים משולבים

גם אם לא התקיים אף תנאי קריטי בודד, ההתרעה תסווג כ-`URGENT` אם מתקיים אחד מהצירופים הבאים:

- `people_count >= 4` וגם `distance_from_fence_m <= 150`
- `people_count >= 3` וגם `vehicle_type == "jeep"`

בכל מקרה אחר, כאשר אף אחד מהתנאים לעיל אינו מתקיים, יש לסווג את ההתרעה כ-`NORMAL`.

כל התרעה חייבת להיבדק מול כלל החוקים לפני קביעת העדיפות הסופית.

Redis – מנגנון תעדוף וניהול תורים

Redis משמש במערכת זו כשכבת תיעדוף מבצעית בזמן אמת. תפקידו לנהל את זרימת ההתרעות לאחר קביעת רמת העדיפות שלהן, ולשמש כמתווך בין שלב הקליטה והסיווג לבין שלב האחסון המתמשך.

המערכת תשתמש ב-Redis כמנגנון תורים (Queue), כאשר ההתרעות יופרדו לשני תורים לוגיים נפרדים:

- `urgent_queue` – עבור התרעות המסווגות כ-URGENT
- `normal_queue` – עבור התרעות המסווגות כ-NORMAL

לאחר קביעת השדה `priority`, כל התרעה תוזרק לתור המתאים לה בהתאם לסיווגה. בכך מבוצעת הפרדה ברורה בין אירועים הדורשים טיפול מיידי לבין אירועים שגרתיים.

רכיב ה-Consumer – שכבת עיבוד ואחסון מתמשך

רכיב ה-Consumer מהווה את שכבת העיבוד השנייה במערכת, ואחראי על משיכת ההתרעות מ-Redis ושמירתן במסד הנתונים MongoDB לצורכי תיעוד היסטורי והמשך ניתוח עתידי.

ה-Consumer פועל באופן רציף ומאזין לשני התורים ב-Redis. מנגנון השליפה חייב לשמור על סדר עדיפות מבצעי, כך שהמערכת תנסה תחילה למשוך התרעות מהתור הדחוף (`urgent_queue`). רק כאשר אין התרעות ממתינות בתור זה, תתבצע שליפה מהתור הרגיל (`normal_queue`).

לאחר שליפת התרעה מ-Redis, ה-Consumer אחראי על ביצוע הפעולות הבאות:

- הוספת שדה חדש בשם `insertion_time`, המייצג את זמן הכנסת הרשומה למסד הנתונים.
- שמירת ההתרעה ב-MongoDB, ללא שינוי שמות השדות המקוריים.

בעת הכנסת הרשומה ל-MongoDB, ייווצר באופן אוטומטי השדה `_id`, אשר ישמש כמזהה הייחודי של ההתרעה במערכת. אין ליצור מזהה ייחודי באופן ידני ואין להוסיף מזהה בקלט המקורי.

רכיב זה אינו מבצע חישוב מחדש של רמת העדיפות ואינו משנה את ערך ה-`priority` שנקבע בשלב הקודם.

שרת אנליטי – REST API (FastAPI)

עליכם לממש שרת REST API מבוסס FastAPI המשמש כשכבת האנליטיקה של המערכת.
השרת מתחבר ל-MongoDB ומספק מענה לשאלות מבצעיות באמצעות קריאה בלבד (Read-Only).

כל Endpoint מייצג שאלה אנליטית ממוקדת ומחזיר תשובה בפורמט JSON תקני וברור.
אין לבצע שינויי נתונים דרך שרת זה.

יש לשמור על הפרדת אחריות:

- שכבת ראוטים נפרדת משכבת הגישה לנתונים (DAL / Repository).
- אין לכתוב לוגיקת מסד נתונים ישירות בתוך הראוטים.
- הנתונים נשלפים מהקולקשן שבו נשמרו ההתרעות.

שימוש ב-Redis כ-Cache Layer (חובה)

שרת האנליטיקה חייב להשתמש ב-Redis כשכבת Cache לתוצאות השאילתות (מודל Cache-Aside).

בכל קריאה ל-Endpoint:

1. תחילה יש לבדוק האם קיימת תוצאה שמורה ב-Redis.
2. אם קיימת — יש להחזיר אותה ישירות ללקוח.
3. אם אינה קיימת — יש להריץ את השאילתה מול MongoDB, לשמור את התוצאה ב-Redis, ולהחזיר אותה.

יש להגדיר TTL סביר עבור נתוני ה-Cache.
כל Endpoint חייב לפעול לפי מנגנון זה, ללא עקיפה של שכבת ה-Cache.

Endpoints נדרשים

להלן השאלות האנליטיות שעליכם לממש כ-Endpoints בשרת:

Route 1

GET /analytics/alerts-by-border-and-priority

שאלה עסקית

מהי ההתפלגות של התרעות לפי גבול ורמת עדיפות?

המטרה:
לספק תמונה ברורה של היקף הפעילות בכל גבול, תוך הבחנה בין התרעות דחופות לבין התרעות רגילות.

התוצאה צריכה לאפשר להבין:

- באילו גבולות נרשמה פעילות חריגה
- היכן כמות האירועים הדחופים גבוהה יחסית

Route 2

GET /analytics/top-urgent-zones

שאלה עסקית

מהם חמשת האזורים (zones) שבהם נרשמה כמות ההתרעות הדחופות הגבוהה ביותר?

המטרה:
לזהות מוקדי פעילות מבצעית משמעותיים הדורשים תשומת לב.

התוצאה צריכה לשקף דירוג ברור של האזורים לפי כמות התרעות URGENT.

Route 3

GET /analytics/distance-distribution

שאלה עסקית

כיצד מתפלגות ההתרעות לפי טווחי מרחק מהגדר?

המטרה:
להבין את עומק החדירה או הקרבה לאזור הרגיש.

יש להציג התפלגות לפי טווחי מרחק מוגדרים, כך שניתן יהיה לראות:

- כמה אירועים התרחשו קרוב מאוד לגדר
- כמה התרחשו במרחק בינוני
- וכמה התרחשו רחוק יותר

Route 4

GET /analytics/low-visibility-high-activity

שאלה עסקית

אילו אזורים מאופיינים במספר גבוה של התרעות שבהן זוהו מספר אנשים משמעותי בתנאי ראות נמוכים?

המטרה:
לאתר אזורים בהם קיימת פעילות חשודה בתנאי תצפית בעייתיים.

התוצאה צריכה לאפשר לזהות אזורים שבהם מתקיימת פעילות משמעותית למרות visibility נמוך.

Route 5

GET /analytics/hot-zones

שאלה עסקית

אילו אזורים מהווים "מוקדי סיכון" – אזורים שבהם קיימת גם כמות גבוהה של התרעות דחופות וגם קרבה יחסית לגדר?

המטרה:
לזהות אזורים שבהם מתקיימת פעילות חוזרת בעלת פוטנציאל סיכון מוגבר.

התוצאה צריכה לשקף אזורים העונים לשני הקריטריונים:

- פעילות דחופה בהיקף משמעותי
- ממוצע מרחק נמוך יחסית מהגדר

דרישות פלט

- כל Endpoint חייב להחזיר JSON קריא וברור.
- אין להחזיר שדות שאינם רלוונטיים לשאלה.
- אין להחזיר את כל המסמכים הגולמיים.
- יש להחזיר נתונים מסוכמים / מנותחים בהתאם לשאלה.

Docker Compose – דרישת הרצה (חובה)

עליכם לארוז את כלל רכיבי המערכת תחת קובץ `docker-compose.yml` אחד מרכזי.

המערכת כולה חייבת לרוץ באמצעות Docker Compose בלבד, ללא הרצה ידנית של רכיבים נפרדים.
יש לוודא שהפרויקט עולה בהצלחה באמצעות:

docker compose up --build

בסיום ההרצה, כל רכיבי המערכת צריכים לפעול ולתקשר ביניהם באופן תקין מקצה לקצה.

להלן מבנה תיקיות נדרש לפרויקט:

```
border-alerts-system/
│
├── docker-compose.yml
├── .env
│
├── producer/
│   ├── Dockerfile
│   ├── main.py
│   ├── priority_logic.py
│   ├── redis_connection.py
│   └── requirements.txt
│
├── consumer/
│   ├── Dockerfile
│   ├── main.py
│   ├── redis_connection.py
│   ├── mongo_connection.py
│   └── requirements.txt
│
├── analytics-api/
│   ├── Dockerfile
│   ├── main.py
│   ├── routes.py
│   ├── dal.py
│   ├── mongo_connection.py
│   ├── redis_connection.py
│   └── requirements.txt
│
└── data/
    └── border_alerts.json
```

דגשים מבניים (חובה)

- קובץ `docker-compose.yml` חייב להימצא בתיקיית ה־Root של הפרויקט בלבד.
  אין למקם אותו בתוך אחת מתיקיות הרכיבים.

- יש להיצמד למבנה הקבצים המוגדר במדויק.
  אין להשמיט קבצים נדרשים ואין לאחד קבצים לתוך קובץ יחיד.

- בכל אחד מהרכיבים נדרשת הפרדה ברורה בין:

  - קובץ `main.py` האחראי על הרצת הרכיב.
  - קבצי חיבור (connection).
  - קבצי לוגיקה / פונקציות עזר.
- אין לערבב לוגיקת חיבור למסד, לוגיקת עיבוד ו־entry point באותו קובץ.

מבנה מסודר והפרדת אחריות הם חלק מדרישות המימוש ומהווים קריטריון להערכה.

עבודה עם Git ו-Branches (חובה)

ניהול גרסאות תקין הוא חלק מדרישות המבחן ומהווה רכיב בציון.

עליכם לעבוד לפי המבנה הבא:

- **main** – נשאר נקי מפיתוח ישיר.
  אין לבצע עליו עבודה שוטפת ואין לבצע אליו commit ישיר.
- **dev** – ענף האינטגרציה המרכזי.
  כל הפיצ'רים ימוזגו אליו לפני מעבר ל-main.
- ענף נפרד לכל רכיב:
  - `feature/producer`
  - `feature/consumer`
  - `feature/api`

כל פיתוח יתבצע אך ורק בענף ה-feature המתאים.
לאחר סיום העבודה על רכיב – יש לבצע merge ל-dev.
בסיום חיבור כלל הרכיבים – יש לבצע merge מ-dev ל-main.

חשוב

- הבדיקה הסופית תתבצע על הקוד שנמצא ב-main בלבד.
  רק קוד שנמצא ב-main ייבדק ויקבל ציון פונקציונלי.
- עם זאת, קיום כל הענפים הנדרשים ייבדק.
  חוסר בענף (dev או אחד מענפי ה-feature) יגרור הורדת ניקוד.
- תהליך עבודה מסודר (feature → dev → main) הוא חלק מדרישות המימוש.